

```
from flask import Flask, jsonify
from sqlalchemy import select
```

 ChatGPT   Mettre à niveau

```
# Import de la base de données et du modèle Courses
from database import db, Courses

# Configuration de l'application
from config import Config

# Fonctions utilitaires
from utils import get_error, get_json_body,

Afficher plus 
```

 ...

Ce code constitue le point d'entrée principal d'une API REST Flask pour gérer des **courses**. Voici une explication de chaque partie.

1. Import des bibliothèques

 Python



 Exécuter

```
from flask import Flask, jsonify
from sqlalchemy import select
```

- Flask : crée l'application web.
- jsonify : convertit les données Python en réponse JSON.
- select : permet d'effectuer des requêtes SQLAlchemy.

2. Import des modules du projet

 Python



 Exécuter

```
from database import db, Courses
from config import Config
from utils import get_error, get_json_body, validate_course_body
from routes.courses import course_bp
from routes.participants import participant_bp
```

- `db` : instance SQLAlchemy.
 - `Courses` : modèle représentant la table des courses.
 - `Config` : configuration de l'application.
 - `get_json_body()` : récupère le JSON envoyé par le client.
 - `validate_course_body()` : vérifie que les données sont valides.
 - `get_error()` : renvoie une erreur au format JSON.
 - `course_bp` et `participant_bp` : Blueprints qui regroupent les routes.
-

3. Création de l'application

</> Python



▶ Exécuter

```
app = Flask(__name__)
```

Création de l'application Flask.

4. Chargement de la configuration

</> Python



▶ Exécuter

```
app.config.from_object(Config)
```

Charge les paramètres définis dans `Config`, par exemple :

- URL de la base de données
 - Mode debug
 - Clé secrète
-

5. Initialisation de SQLAlchemy

</> Python



▶ Exécuter

```
db.init_app(app)
```

Relie la base de données à l'application Flask.

6. Création des tables

</> Python



▶ Exécuter

```
with app.app_context():  
    db.create_all()
```

Crée automatiquement les tables de la base si elles n'existent pas.

7. Enregistrement des Blueprints

</> Python



▶ Exécuter

```
app.register_blueprint(course_bp)  
app.register_blueprint(participant_bp)
```

Permet d'ajouter les routes définies dans d'autres fichiers.

Exemple :

- routes/courses.py
 - routes/participants.py
-

Route GET /courses

</> Python



▶ Exécuter

```
@app.get("/courses")  
def list_courses():
```

Cette route renvoie toutes les courses.

Requête SQL

</> Python



▶ Exécuter

```
statement = select(Courses)
```

Équivalent SQL :

</> SQL



```
SELECT * FROM courses;
```

Exécution

</> Python



▶ Exécuter

```
courses = db.session.scalars(statement).all()
```

Récupère tous les objets Courses .

Retour JSON

</> Python



▶ Exécuter

```
return jsonify([course.to_dict() for course in courses])
```

Chaque objet est converti en dictionnaire grâce à `to_dict()` .

Exemple :

</> JSON



```
[
  {
    "id": 1,
    "nom": "Marathon",
    "distance": 42
  },
  {
    "id": 2,
    "nom": "Semi",
    "distance": 21
  }
]
```

Route GET /courses/<id>

</> Python



▶ Exécuter

```
@app.get("/courses/<int:course_id>")
```

Permet de récupérer une seule course.

Exemple :

GET /courses/5



Recherche

</> Python



Exécuter

```
course = db.session.get(Courses, course_id)
```

Équivalent SQL :

</> SQL



```
SELECT * FROM courses  
WHERE id = 5;
```

Si la course n'existe pas

</> Python



Exécuter

```
if course is None:
```

Retour :

</> JSON



```
{  
    "error": "Cours introuvable"  
}
```

avec le code HTTP :

404 Not Found



Sinon

</> Python



Exécuter

```
return jsonify(course.to_dict())
```

Retour :

</> JSON



```
{
  "id": 5,
  "nom": "Trail",
  "distance": 15
}
```

Route POST /courses

</> Python



Exécuter

```
@app.post("/courses")
```

Ajoute une nouvelle course.

Lecture du JSON

</> Python



Exécuter

```
body = get_json_body()
```

Exemple reçu :

</> JSON



```
{
  "nom": "Marathon",
  "distance": 42
}
```

Validation

</> Python



Exécuter

```
validation_error = validate_course_body(body)
```

Vérifie par exemple :

- que nom existe ;
- que distance est présente ;

- que les types sont corrects.

Si erreur

</> Python



▶ Exécuter

```
if validation_error:
    return get_error(validation_error)
```

Réponse :

</> JSON



```
{
  "error": "Le champ distance est obligatoire"
}
```

Création de l'objet

</> Python



▶ Exécuter

```
course = Courses()
```

Création d'une nouvelle instance du modèle.

Remplissage

</> Python



▶ Exécuter

```
course.from_body(body)
```

Copie les valeurs du JSON dans l'objet.

Par exemple :

</> Python



▶ Exécuter

```
course.nom = body["nom"]
course.distance = body["distance"]
```

Ajout à la session

</> Python



▶ Exécuter

```
db.session.add(course)
```

Prépare l'insertion dans la base.

Validation

</> Python



▶ Exécuter

```
db.session.commit()
```

Envoie réellement la requête SQL.

Équivalent :

</> SQL



```
INSERT INTO courses (...)  
VALUES (...);
```

Retour

</> Python



▶ Exécuter

```
return jsonify(course.to_dict()), 201
```

Code HTTP :

201 Created



Exemple :

</> JSON



```
{  
  "id": 8,  
  "nom": "Marathon",  
  "distance": 42  
}
```

Lancement de l'application

</> Python



Exécuter

```
if __name__ == "__main__":  
    app.run(debug=True)
```

Ce bloc ne s'exécute que lorsque le fichier est lancé directement.

Il démarre le serveur Flask en mode développement :

- rechargement automatique du code ;
- affichage détaillé des erreurs.

Résumé des routes

Méthode	URL	Action	Code de succès
GET	/courses	Liste toutes les courses	200
GET	/courses/<id>	Retourne une course	200
POST	/courses	Crée une nouvelle course	201

Cette structure respecte une organisation classique d'une API Flask : l'application est configurée dans `app.py` , la base de données est initialisée avec SQLAlchemy, les routes sont réparties en Blueprints, et les modèles (`Courses`) assurent la conversion entre les données de la base et les réponses JSON.

